AnoBot – Full Project Specification

A Roboflow-style computer vision platform: data upload, labeling, dataset versions, model training, testing and deployment, with admin-managed user accounts. This document is written to be pasted into Claude (Claude Code recommended) to build both the backend and the frontend.

1. How to use this document

Paste the whole document into Claude and add this instruction:

> Read the specification below. Build the project in the phases listed in section 17, one phase at a time. After each phase, run the tests, show me how to start the app, and wait for my approval before starting the next phase. Ask me before changing any decision in this document.

A working front-end prototype already exists (single HTML file, in-memory data, simulated training). It is the visual and behavioral reference: https://claude.ai/artifact/66YG7RogajVwng2TTDRgKX. Anything in this document overrides the prototype where they differ.

2. Product overview

AnoBot lets a team build custom computer vision models without writing training code.

Workflow: upload images, label them, generate a dataset version (split + preprocessing + augmentation), train a model, test it, deploy it through an API, and export datasets in standard formats.

Users: accounts are created only by an administrator (no public sign-up). There are two roles: **admin** and **user**.

Goals

- Cover the core Roboflow workflow end to end with real training and real hosted inference, not simulations.
- Support five project types: object detection, classification, instance segmentation, keypoint detection and oriented bounding boxes.
- Provide an admin area for users, API keys, activity log and settings.

- Use Gemini (through the backend) for AI-assisted labeling, class suggestions, training advice and test inference.

Non-goals for version 1: public sign-up, billing, public dataset marketplace with real community uploads, semantic-segmentation mask painting, multi-tenant organizations.

3. Recommended technology stack

Layer	Choice	Notes
Frontend	React 18 + TypeScript + Vite	Tailwind CSS for styling, React Router, TanStack Query, Zustand for UI state
Canvas labeling	HTML canvas (own component) or Konva	Must handle boxes, polygons, keypoints, rotated boxes
Backend	Python 3.11 + FastAPI	Async, OpenAPI docs auto-generated
Database	PostgreSQL 16	SQLAlchemy 2 + Alembic migrations
Queue / workers	Redis + RQ (or Celery)	Training, auto-labeling batches, exports
Object storage	S3-compatible (MinIO locally)	Images, thumbnails, dataset exports, model weights
Training	Ultralytics (YOLOv8/YOLO11)	Detection, segmentation, classification, pose, OBB all supported
Inference serving	FastAPI worker loading weights (ONNX or PyTorch)	GPU optional, CPU fallback
AI assist	Google Gemini REST API	Called from the backend only
Auth	Argon2id password hashes, JWT in httpOnly cookies	Refresh-token rotation
Packaging	Docker Compose	api, worker, web, postgres, redis, minio

If a different stack is preferred, keep the same API contract (section 9).

4. Roles and permissions

Capability	Admin	User
Sign in, change own password	Yes	Yes
Create projects, upload, label, version, train, test, deploy, export	Yes	Yes
Delete own projects	Yes	Yes
Delete any project	Yes	No
Manage users (create, edit, disable, remove, reset password)	Yes	No
Manage API keys (Gemini slots)	Yes	No
View activity log	Yes	No
Workspace settings, backup and restore	Yes	No

Rules:

- There is no sign-up page. On first run (no users in the database) the app shows **Create admin account**; that account becomes the first admin.
- The last active admin can never be removed, demoted or disabled. An admin cannot remove or disable their own account.
- Passwords: minimum length is configurable (default 8). Store only Argon2id hashes.
- Five failed sign-ins for one username or IP lock sign-in for 30 seconds.
- Every authorization check happens on the server. The frontend only hides what the user cannot use.

5. Project types

A project has exactly one type, chosen at creation (admin or user can change it only while the project has no annotations).

Type	Annotation	Training task	Export formats
Object detection	Axis-aligned boxes	YOLO detect	COCO JSON, YOLO TXT, Pascal VOC, CSV
Classification	One class per image	YOLO classify	Folder-per-class, CSV
Instance segmentation	Polygons	YOLO segment	COCO segmentation, YOLO-seg TXT
Keypoint detection	Named ordered keypoints with visibility	YOLO pose	COCO keypoints, YOLO-pose TXT
Oriented bounding boxes	Rotated boxes (center, width, height, angle)	YOLO obb	YOLO-OBB TXT, DOTA TXT

Coordinates are stored **normalized (0–1)** relative to the original image. Rotation angle is stored in degrees, range −90 to 90.

6. Frontend specification

6.1 Visual design

- White theme only (no dark mode). Background and surfaces are white, borders light gray (`#e5e5e5`), body text near-black (`#1a1a1a`), muted text `#666` .
- Brand accent: purple `#6706ce` , light tint `#f1e6fc` . Success `#12a36b` , error `#d93a52` .
- Font: Inter, with system-ui fallback.
- Layout like Roboflow: a 52 px top bar, a 224 px left sidebar, content area with 22 px padding and a 1100 px max width. On screens narrower than 760 px the sidebar becomes a horizontally scrolling tab strip above the content.
- Top bar (left to right): logo `◈ AnoBot` , breadcrumb `workspace / project name` (project name is editable inline), project-type selector (a dropdown styled as a purple badge), then on the right the signed-in user name, a role badge, an **Account** button and a **Sign out** button.
- Cards: white, 1 px border, 12 px radius, 16 px padding.

6.2 Sidebar navigation

Groups and items (the labeling item changes with the project type):

- **Project:** Overview, Upload Data, *labeling tool*, Versions, Health Check
- **Models:** Train, Visualize, Video
- **Deploy:** Deployments, Workflows, Export Dataset
- **Admin** (admins only): Users, API Keys, Activity Log, Settings
- **Explore:** Universe

Labeling item by project type: Annotate (detection), Classify (classification), Polygons (instance segmentation), Keypoints (keypoint detection), Rotated Boxes (oriented boxes). A group with no visible items is hidden entirely.

6.3 Screens

Sign-in / first-run screen. Centered card with the logo. Fields: username, password (plus full name on first run). Enter key submits. Show "Invalid username or password" without revealing which one is wrong. Show a lockout message after repeated failures.

Overview. Welcome card with a five-step progress bar (Upload images, Annotate, Generate a version, Train a model, Deploy). Stats strip: images, annotated images, annotations, classes. One card per step with status (Done or To do) and a Start or Review button that navigates to that screen.

Upload Data. Dashed drag-and-drop zone plus a Select images button; multiple files; progress per file; server-side thumbnails. Class manager: chips with a delete cross, an add-class input, and an **Suggest classes (Gemini)** button. Image grid with a badge showing the annotation count per image; clicking an image opens it in the labeling tool. Filters: all, labeled, unlabeled, by class. Bulk delete.

Labeling tools (shared behavior). Image on a canvas that scales to the container width; previous and next buttons and arrow-key navigation; a class selector (keys 1–9 pick a class); a side list of annotations with delete; autosave to the server on every change (debounced 500 ms). Undo and redo (Ctrl+Z, Ctrl+Shift+Z). Zoom and pan (mouse wheel and space-drag). Each tool:

- **Annotate (boxes):** drag to draw; resize and move handles; **Auto-label (Gemini)** button that adds suggested boxes the user can accept or reject.
- **Polygons:** click to place points, Enter or a button closes the shape, undo last point, edit vertices by dragging.

- **Classify:** grid of images, each with a class dropdown; keyboard shortcuts; **Auto-classify (Gemini)**.
- **Keypoints:** an editable schema (ordered, comma-separated names, plus optional skeleton edges); click places the next keypoint; **Not visible** button marks it hidden (visibility 0); the status line says which keypoint is next.
- **Rotated boxes:** drag to draw a box, then a rotation slider (−90° to 90°) per box, or a rotation handle on the canvas.

Versions. Form: train/valid/test percentages (test is the remainder), resize (416, 640, 1024), preprocessing (grayscale, auto-orient), augmentations (horizontal flip, rotation ±15°, brightness ±20%, blur, mosaic) with a copies-per-image multiplier (1–5). Creating a version is a background job. Table of versions with counts, settings and status (queued, building, ready, failed).

Health Check. Class balance bar chart, counts of unlabeled images, tiny boxes, empty classes, image size distribution, and plain-language recommendations (for example: fewer than 50 images, class imbalance above 5x).

Train. Pick a version, model family and size (YOLO n/s/m/l, with the task matching the project type), epochs, batch size, image size, and a **start from** option (COCO-pretrained or a previous model). Live metrics over a WebSocket or server-sent events: epoch, loss curves, precision, recall, mAP50, mAP50-95, GPU/CPU usage, log tail. **Stop** button. **Training advisor (Gemini)** button returns advice from dataset stats. Finished runs appear in a model list with metrics, a confusion matrix and a PR curve.

Visualize. Pick a trained model (or Gemini as a fallback), upload an image, set a confidence threshold slider, see predictions drawn for the project type (boxes, masks or polygons, keypoints, rotated boxes, class label).

Video. Upload a video or use the webcam; run the model every N frames; overlay predictions; show per-class counts over time.

Deployments. For each model: a hosted inference endpoint with copyable cURL, Python and JavaScript snippets (using a per-project API key), an API-key manager (create, copy once, revoke), usage counters, and downloads of model weights in PyTorch, ONNX and TFLite formats.

Workflows. A block pipeline builder. Blocks: Input image, Object detection (selected model), Filter by class, Count objects, Crop detections, Ask Gemini (visual question), Send alert (webhook). Blocks can be reordered and removed; each has parameters. A **Run**

button executes the workflow on an uploaded image and shows the result log and drawn detections. Workflows can be saved, named and exposed as an API endpoint.

Export Dataset. Choose a version and a format from the table in section 5; the server builds a ZIP in the background and offers a download link.

Universe. A browsable catalog of public datasets with search, filters by project type, image counts and a **Fork** button that copies the dataset (images, labels, classes) into a new project. Seed it with a few real open datasets (with correct licenses) and let admins mark any project as public.

Users (admin). Create form: full name, username, role (admin or user), temporary password with a Generate button. Table: name, username, role, status, last sign-in, Edit and Remove. Edit form: name, role, active toggle, optional new password. Remove needs a second confirmation click. Show the temporary password once after creation.

API Keys (admin). Four Gemini key slots (see section 11). For each slot: masked input, Show, Save, Test, Clear. The server stores keys encrypted and never returns the full key after saving (only the last four characters). Test makes a minimal Gemini call and reports success or the error.

Activity Log (admin). Table of time, user, action, with filters by user, action type and date range, pagination, and a **Clear log** action (itself logged).

Settings (admin). Workspace name, minimum password length, session lifetime, allowed upload size and file types, default Gemini model name, role permission reference table, backup and restore.

My account (everyone). Profile summary and a change-password form (current, new, confirm).

6.4 Frontend behaviors

- Route guard: unauthenticated users go to sign-in; users who open an admin route get a 403 screen or are redirected to Overview.
- All forms show inline success and error messages.
- Long jobs (training, version build, export, bulk auto-label) show progress and survive page reloads.
- Keyboard accessible; visible focus styles; labels on every input.

7. Backend specification

7.1 Services

- **api:** FastAPI application with routers for auth, users, projects, images, annotations, versions, models, training, inference, workflows, exports, universe, admin and gemini.
- **worker:** RQ or Celery worker running version builds, exports, training and batch auto-labeling. GPU workers optional; training jobs are routed to a GPU queue when available.
- **inference:** an inference service (can live inside the api process for version 1) that loads model weights from storage, caches them in memory (LRU), and serves predictions.
- **postgres, redis, minio:** supporting services.

7.2 Image handling

- Accept JPEG, PNG, WebP, BMP; reject other types by content sniffing, not just extension. Default max 25 MB per file and 5000 images per upload batch (configurable).
- Strip EXIF after applying orientation. Store the original, a 320 px thumbnail and a 1280 px preview. Record width and height.
- Compute a perceptual hash to warn about duplicates.

7.3 Dataset version build

1. Collect labeled images (and unlabeled images flagged as null examples if the user opts in).
2. Assign splits with a seeded shuffle so a version is reproducible; stratify by class where possible.
3. Apply preprocessing (resize, grayscale, auto-orient) to all splits.
4. Apply augmentations **only to the training split**, writing new images and transformed annotations (boxes, polygons, keypoints and rotated boxes must all be transformed correctly, including flips that swap left/right keypoint names).
5. Write the dataset in the Ultralytics layout (`images/{train,val,test}` , `labels/...` , `data.yaml`) and store a manifest in the database.

7.4 Training

- Training runs the Ultralytics Python API in a worker process with the task derived from the project type.

- Stream per-epoch metrics to Redis; the API relays them to the browser over WebSocket or server-sent events and also writes them to the database.
- Save best and last weights to object storage. Export ONNX automatically after training; TFLite on request.
- Support cancel (terminate the process cleanly), resume from last weights and a configurable timeout.
- Enforce one active training job per user by default (configurable).

7.5 Inference

- `POST /api/v1/infer/{model_id}` accepts an image (multipart or base64) and optional `confidence` and `iou`. Returns predictions in a task-specific JSON shape (section 9.3).
- Per-project API keys authorize inference calls. Keys are shown once, stored as hashes, and can be revoked. Count requests per key per day.

7.6 Gemini proxy

The browser never sees Gemini keys. The backend reads the key for the requested slot, decrypts it in memory, calls the Gemini REST API and returns a normalized result. See section 11 for the four slots and prompts. Apply per-user rate limits and timeouts, and log failures (without logging keys or images).

8. Data model

Use UUID primary keys and `created_at` / `updated_at` timestamps on every table.

- **users**: id, name, username (unique, lowercase), role (`admin` or `user`), is_active, password_hash, last_login_at.
- **sessions** (or refresh_tokens): id, user_id, token_hash, expires_at, user_agent, ip.
- **settings**: key, value (JSON). Keys include workspace_name, min_password_length, session_hours, max_upload_mb, gemini_model.
- **api_keys_gemini**: slot (`autoLabel` , `classSuggest` , `advisor` , `inference`), ciphertext, last4, updated_by.
- **audit_log**: id, user_id, username, action, target_type, target_id, metadata (JSON), ip, created_at.

- **projects**: id, owner_id, name, type (`detect`, `classify`, `segment`, `keypoint`, `obb`), is_public, keypoint_schema (JSON: names and edges).
- **classes**: id, project_id, name, color, position.
- **images**: id, project_id, filename, storage_key, thumb_key, width, height, phash, split_hint, uploaded_by.
- **annotations**: id, image_id, class_id, kind (`box`, `polygon`, `keypoints`, `obb`, `tag`), geometry (JSON, normalized), created_by, source (`manual` or `gemini`).
- **versions**: id, project_id, number, settings (JSON), counts (JSON), status, manifest_key, error.
- **models**: id, project_id, version_id, name, architecture, task, status, metrics (JSON), weights_key, onnx_key, created_by.
- **training_runs**: id, model_id, params (JSON), status, started_at, finished_at, log_key, epoch_metrics (JSON array).
- **deploy_keys**: id, project_id, name, key_hash, last4, revoked_at, created_by.
- **usage_counters**: key_id, day, requests.
- **workflows**: id, project_id, name, blocks (JSON), is_api_enabled.
- **exports**: id, version_id, format, status, storage_key, expires_at.
- **universe_datasets**: id, source_project_id, title, description, license, tags, image_count.

Geometry JSON shapes (all normalized to 0–1):

```
{ "box":     { "x": 0.1, "y": 0.2, "w": 0.3, "h": 0.4 },
  "polygon": { "points": [[0.1,0.2],[0.3,0.2],[0.2,0.5]] },
  "keypoints": { "points": [[0.5,0.1,2],[0,0,0]] },
  "obb":     { "cx": 0.5, "cy": 0.5, "w": 0.3, "h": 0.2, "angle": 30 },
  "tag":     {} }
```

For keypoints the third value is visibility: 0 not labeled, 1 labeled but occluded, 2 visible.

9. REST API contract

Base path `/api/v1`. JSON in and out. Errors use `{ "error": { "code": "...", "message": "..." } }` with proper HTTP status codes. Authentication is a session cookie (httpOnly, SameSite=Lax, Secure in production) for the web app, and `Authorization: Bearer <deploy key>` for inference endpoints.

9.1 Auth and users

Method	Path	Who	Purpose
GET	/auth/status	public	Returns `{ needs_setup: bool }`
POST	/auth/setup	public, only if no users	Create the first admin
POST	/auth/login	public	Username and password sign-in
POST	/auth/logout	any	End session
GET	/auth/me	any	Current user
POST	/auth/change-password	any	Current and new password
GET	/users	admin	List users
POST	/users	admin	Create user (name, username, role, password)
PATCH	/users/{id}	admin	Edit name, role, active, optional new password
DELETE	/users/{id}	admin	Remove user (guards: not self, not last admin)

9.2 Projects, data and labeling

Method	Path	Purpose
GET, POST	/projects	List and create projects
GET, PATCH, DELETE	/projects/{id}	Read, rename, change type, delete
GET, POST	/projects/{id}/classes	List and add classes
PATCH, DELETE	/classes/{id}	Rename, recolor, delete
POST	/projects/{id}/images	Multipart upload (many files)
GET	/projects/{id}/images	Paginated list with filters

Method	Path	Purpose
DELETE	/images/{id}	Delete one image
GET	/images/{id}/annotations	All annotations for an image
PUT	/images/{id}/annotations	Replace the annotation set (used by autosave)
GET	/projects/{id}/health	Dataset health report
POST	/projects/{id}/versions	Create a version (background job)
GET	/projects/{id}/versions	List versions
POST	/versions/{id}/exports	Start an export with a format
GET	/exports/{id}	Status and download URL

9.3 Models, training and inference

Method	Path	Purpose
POST	/versions/{id}/train	Start training with architecture, epochs, batch, image size
GET	/models/{id}	Model details and metrics
GET	/models/{id}/stream	Server-sent events with live metrics
POST	/models/{id}/stop	Cancel training
GET	/models/{id}/weights?format=pt|onnx|tflite	Download weights
POST	/infer/{model_id}	Run inference (session or deploy key)
POST, GET, DELETE	/projects/{id}/deploy-keys	Create (shown once), list, revoke
POST, GET, PUT, DELETE	/projects/{id}/workflows	Save and manage workflows
POST	/workflows/{id}/run	Run a workflow on an image

Inference response by project type:

```
{ "detect":   [{ "label": "cat", "confidence": 0.91, "box":
{"x":0.1,"y":0.2,"w":0.3,"h":0.4} }],
  "segment":  [{ "label": "cat", "confidence": 0.88, "polygon": [[0.1,0.2],
[0.3,0.2]] }],
  "classify": [{ "label": "healthy", "confidence": 0.97 }],
  "keypoint": [{ "label": "person", "confidence": 0.9, "keypoints":
[[0.5,0.1,0.95]] }],
  "obb":      [{ "label": "ship", "confidence": 0.8, "obb":
{"cx":0.5,"cy":0.5,"w":0.3,"h":0.1,"angle":25} }] }
```

9.4 Admin and AI

Method	Path	Who	Purpose
GET, PUT	/admin/gemini-keys	admin	List slots (last four characters only), save a key
DELETE	/admin/gemini-keys/{slot}	admin	Clear a key
POST	/admin/gemini-keys/{slot}/test	admin	Test a key
GET	/admin/audit	admin	Filtered, paginated log
DELETE	/admin/audit	admin	Clear log (logged)
GET, PUT	/admin/settings	admin	Read and update settings
GET	/admin/backup	admin	Export users, settings and audit log as JSON
POST	/admin/restore	admin	Restore from a backup file
POST	/ai/suggest-classes	user	Slot `classSuggest`
POST	/ai/auto-label	user	Slot `autoLabel`
POST	/ai/training-advice	user	Slot `advisor`
POST	/ai/detect	user	Slot `inference`
GET	/universe	user	Browse public datasets
POST	/universe/{id}/fork	user	Copy a dataset into a new project

10. Export formats

Implement these exporters and test each with a round trip against a reference tool where possible:

- **COCO JSON** (boxes, segmentation polygons, keypoints with `num_keypoints` and `keypoints` arrays of x, y, visibility).
- **YOLO TXT** (`class cx cy w h`, normalized), plus `data.yaml`.

- **YOLO-seg** (`class x1 y1 x2 y2 ...`), **YOLO-pose** (`class cx cy w h x1 y1 v1 ...`), **YOLO-OBB** (`class x1 y1 x2 y2 x3 y3 x4 y4`, normalized corner points computed from center, size and angle).
- **Pascal VOC XML**, **CSV** (`file,class,x,y,w,h`), **DOTA TXT**, and a **folder-per-class** layout for classification.

11. Gemini integration (four key slots)

The prototype used four placeholder keys. In the real system they are stored encrypted in the database and managed on the API Keys admin page.

Slot	Used by	Request	Expected response
`autoLabel`	Auto-label and auto-classify	Image plus the project's class list	JSON array of `{label, box_2d: [ymin,xmin,ymax,xmax]}` normalized 0–1000, or a single class name for classification
`classSuggest`	Suggest classes on Upload Data	One or more sample images	JSON array of short lowercase category names
`advisor`	Training advisor	Dataset statistics, classes, version settings	Plain-text advice under 120 words
`inference`	Visualize, Workflows, Video fallback	Image plus class list	JSON array of `{label, confidence, box_2d}`

Requirements: instruct Gemini to return JSON only, strip code fences, validate with a schema, convert `box_2d` to normalized `x, y, w, h`, drop invalid items, and mark created annotations with `source = gemini` so users can review them. Use the configured model name (default `gemini-2.0-flash`).

12. Security requirements

- Argon2id for passwords; constant-time comparisons; no password or key in logs.
- httpOnly session cookies with CSRF protection (double-submit token or SameSite plus Origin checks); short access token and rotating refresh token.

- Role checks implemented as FastAPI dependencies on every route; add tests that every admin route returns 403 for a user.
- Gemini keys encrypted with a server-side master key (Fernet or KMS); the master key comes from an environment variable, never from the database.
- Deploy keys stored as hashes; shown once.
- Strict file validation, size limits, pre-signed URLs with short expiry for image access, and no path traversal in storage keys.
- Rate limiting on sign-in, uploads, AI endpoints and inference.
- CORS limited to the configured web origin; security headers (CSP, X-Content-Type-Options, frame protections).
- Audit-log every sign-in, failed sign-in, user change, key change, setting change, project deletion, export and training start.

13. Suggested repository structure

```
anobot/
  docker-compose.yml
  .env.example
  README.md
  backend/
    app/
      main.py
      core/        (config, security, deps, logging)
      models/      (SQLAlchemy models)
      schemas/     (Pydantic models)
      routers/     (auth, users, projects, images, annotations, versions,
                    models, infer, workflows, exports, universe, admin, ai)
      services/    (storage, imaging, augment, exporters, trainer,
                    inference, gemini, audit)
      workers/     (jobs.py, train_job.py)
    alembic/
    tests/
    pyproject.toml
  frontend/
    src/
      app/         (router, layout, guards)
      api/         (typed client, query hooks)
      features/    (auth, overview, upload, labeling/{boxes,polygons,
                    classify,keypoints,obb}, versions, health, train,
```

```
                    visualize, video, deployments, workflows, export,
                    universe, admin/{users,keys,activity,settings}, account)
    components/  (Card, Table, Button, Chip, Canvas helpers)
    styles/
  package.json
```

14. Environment variables

```
DATABASE_URL, REDIS_URL
S3_ENDPOINT, S3_BUCKET, S3_ACCESS_KEY, S3_SECRET_KEY
SECRET_KEY            (session signing)
MASTER_ENCRYPTION_KEY (encrypts Gemini keys)
WEB_ORIGIN            (CORS and cookie settings)
MAX_UPLOAD_MB, DEFAULT_GEMINI_MODEL
TRAINING_DEVICE       (cpu | cuda)
```

15. Quality bar and testing

- Backend: pytest with a test database; unit tests for exporters, geometry transforms (flip, rotate, rotated-box corners), role guards and the last-admin rule; API tests for every router.
- Frontend: component tests for labeling canvases (coordinate math) and end-to-end tests (Playwright) covering first-run setup, sign-in, create user, upload, label, create version, start training (with a tiny dataset and 1 epoch), run inference, export.
- Linting and type checks in CI (ruff, mypy, eslint, tsc).
- Accessibility check on all forms.

16. Known limitations carried over from the prototype

- The prototype stores everything in memory or browser storage, uses simulated training, and cannot call Gemini from its hosted preview. None of that should be copied: use the real backend described here.
- The prototype's client-side login is a demo only. Real authentication must happen on the server.
- Universe in the prototype is a static list with no images; the real one forks real projects.

17. Build phases and acceptance criteria

Phase 1 – Foundation. Docker Compose, database and migrations, config, health endpoint, frontend shell with the white theme, top bar and sidebar. *Accept:* `docker compose up` shows the empty app.

Phase 2 – Auth and admin users. First-run admin setup, sign-in, sign-out, change password, role guards, Users page (create, edit, disable, remove with all guards), audit logging. *Accept:* the last-admin and self-removal rules are enforced and tested; a user receives 403 on every admin route.

Phase 3 – Projects, upload and classes. Project CRUD with type, image upload with thumbnails, class manager, image grid with filters. *Accept:* 500 images upload and display with thumbnails.

Phase 4 – Labeling tools. Boxes, classification, polygons, keypoints and rotated boxes with autosave, undo and keyboard shortcuts. *Accept:* annotations round-trip exactly through the API and reload correctly.

Phase 5 – Versions, health check and exports. Version builder with augmentation, health report, all export formats. *Accept:* exported datasets load in Ultralytics and in a COCO viewer without errors.

Phase 6 – Training and models. Worker queue, real training for all five tasks, live metrics stream, model list with metrics, weights download. *Accept:* a small dataset trains for a few epochs and produces a model with visible metrics.

Phase 7 – Inference, Visualize, Video and Deployments. Inference endpoint, deploy keys, snippets, Visualize and Video screens. *Accept:* a cURL call with a deploy key returns correct predictions for each project type.

Phase 8 – Gemini and API Keys admin page. Encrypted key storage, slot testing, four AI features with schema validation. *Accept:* with valid keys, auto-label, class suggestion, advisor and detect all work; without keys, each fails with a clear message.

Phase 9 – Workflows, Universe, Activity Log, Settings, backup. Remaining admin screens and the workflow runner. *Accept:* backup and restore round-trips users and settings.

Phase 10 – Hardening and release. Rate limits, security headers, end-to-end tests, documentation (README with setup, environment variables and an admin guide). *Accept:* the full end-to-end test passes in CI.

18. Open questions to confirm with the product owner

1. Deployment target: a single GPU server, a cloud GPU service, or CPU only?
2. Should users see only their own projects, or all projects in the workspace? (Default in this spec: users see all projects, can delete only their own.)
3. Is semantic segmentation (pixel masks) needed in version 1?
4. Which public datasets and licenses should seed Universe?
5. Should email-based password reset be added later, or stay admin-only?